Q 1.Defining and calling functions in Python.

A.Defining a Function

We use the def keyword to define a function.

Syntax

```
def function_name():
  # code
```

Example: Defining and Calling Function

```
def text():
  print("Hello, Welcome to Python!")

# Calling the function
text()
```

Output :

Hello, Welcome to Python!

Q 2.Function arguments (positional, keyword, default).

A. 1. Positional Arguments

Arguments are passed in the same order as parameters.

Example :

```
def student(name, age):
  print("Name:", name)
  print("Age:", age)
student("Sonali", 20)
```

Output :

Name: Sonali
Age: 20

2. Keyword Arguments

Arguments are passed using parameter names.

Example :

```
def student(name, age):
```

```
    print("Name:", name)
    print("Age:", age)
student(age=20, name="Sonali")
```

Output :

Name: Sonali
Age: 20

3. Default Arguments

Default values are assigned to parameters.

Example:

```
def TEXT(name="XYZ"):
    print("Hello", name)
greet("Sonali")
greet()
```

Output :

Hello Sonali
Hello XYZ

Q 3.Scope of variables in Python.

A.Scope means where a variable can be used in a program.
Python has mainly 2 types of variable scope:

- Local Scope
- Global Scope

1. Local Scope

A **local variable** is created **inside a function** and can only be used **inside that function**.

Example

```
def my_function():

    x = 10   # Local variable

    print("Inside function:", x)
```

```
my_function()
```

Output

```
Inside function: 10
```

2. Global Scope

A **global variable** is created **outside the function** and can be used **anywhere**.

Example

```
x = 20   # Global variable

def my_function():

   print("Inside function:", x)

my_function()

print("Outside function:", x)
```

Output

```
Inside function: 20

Outside function: 20
```

Example with Both

```
x = 5   # Global

def test():

   x = 10   # Local

   print("Local:", x)

test()

print("Global:", x)
```

Output

Local: 10

Global: 5

Q 4.Built-in methods for strings, lists, etc.

A. String Methods

B. Strings are immutable (cannot change original value)

C. Common String Methods

D. text = "python programming"

Method	Description	Example
`upper()`	Convert to uppercase	`text.upper()`
`lower()`	Convert to lowercase	`text.lower()`
`title()`	Capitalize words	`text.title()`
`capitalize()`	First letter capital	`text.capitalize()`

`replace()`	**Replace text**	`text.replace("python","Java")`
`find()`	**Find position**	`text.find("pro")`
`count()`	**Count words**	`text.count("p")`
`strip()`	**Remove spaces**	`" hello ".strip()`
`split()`	**Split string**	`text.split()`

Example

```
text = "hello python"
print(text.upper())
print(text.replace("python","world"))
```

Output

```
HELLO PYTHON
hello world
```

List Methods

Lists are mutable (can change values)

```
numbers = [1, 2, 3]
```

Method	Description
append()	Add element
insert()	Insert at position
remove()	Remove element
pop()	Remove last element
sort()	Sort list
reverse()	Reverse list
count()	Count items
index()	Find position
extend()	Add multiple items

Example

```
numbers = [1, 2, 3]
numbers.append(4)
numbers.remove(2)
```

```
print(numbers)
```

Output

```
[1, 3, 4]
```

Dictionary Methods

```
student = {"name": "Sonali", "age": 20}
```

Method	Description
keys()	Get keys
values()	Get values
items()	Get key-value pairs
update()	Update dictionary
pop()	Remove item
get()	Get value safely

Example

```
print(student.keys())
print(student.values())
```

Set Methods

s = {1, 2, 3}

Method	Description
add()	Add element
remove()	Remove element
union()	Combine sets
intersection()	Common elements
difference()	Unique elements
clear()	Remove all

Q 5.Understanding how generators work in Python.

A.A generator is a special type of function that returns values one by one using the yield keyword instead of return.

Generator Function (yield)

```
def test():
    yield "Hello"
```

```
    yield "Python"

k = test()

print(next(k))

print(next(k))
```

Q 6.Difference between yield and return.

A.return

- Ends the function completely
- Returns a single value
- Function stops after return

Example :

```
def test():

   return 1

   return 2

print(test())
```

Output :

```
1
```

yield

- Used in **generator functions**
- Returns values **one by one**
- Function **pauses**, not ends

Example :

```
def test():

   yield 1

   yield 2
```

```
    yield 3
for i in test():
    print(i)
```

Output :

1

2

3

Q 7.Understanding iterators and creating custom iterators.

A.An iterator is an object that allows you to loop through values one by one.

Example of Iterator

```
numbers = [1, 2, 3, 4]
for i in numbers:
    print(i)
```

Output

1

2

3

4

Creating Custom Iterator

You can manually create an iterator using **iter()** and **next()**

```
numbers = [10, 20, 30]
it = iter(numbers)
```

```
print(next(it))

print(next(it))

print(next(it))
```

Output

10

20

30

Q 8.How functional programming works in Python.

A.Functional Programming means writing programs using functions instead of changing data directly.

Python supports functional programming using:

- Functions as variables
- map()
- filter()
- reduce()
- Lambda functions

1. Functions as Variables

In Python, functions can be assigned to variables.

```
def greet():

  print("Hello")

x = greet

x()
```

Output :

Hello

2. Lambda Function (Anonymous Function)

Lambda is a **small one-line function**

```
square = lambda x: x * x

print(square(5))
```

Output

25

3. map() Function

map() applies a function to all elements in a list

```
numbers = [1, 2, 3, 4]

result = map(lambda x: x * 2, numbers)

print(list(result))
```

Output :

[2, 4, 6, 8]

4. filter() Function

filter() selects elements based on condition

```
numbers = [1, 2, 3, 4, 5, 6]

result = filter(lambda x: x % 2 == 0, numbers)

print(list(result))
```

Output :

[2, 4, 6]

5. reduce() Function

reduce() reduces list to single value

```
from functools import reduce

numbers = [1, 2, 3, 4]

result = reduce(lambda x, y: x + y, numbers)

print(result)
```

Output :

10

Q 9.Using map(), reduce(), and filter() functions for processing data.

A.1. map() Function

map() applies a function to each element of a list.

Syntax :

```
map(function, iterable)
```

Example: Square Numbers

```
numbers = [1, 2, 3, 4, 5]

result = map(lambda x: x*x, numbers)

print(list(result))
```

Output :

[1, 4, 9, 16, 25]

Each number is squared

2. filter() Function

filter() selects elements based on a condition

Syntax :

```
filter(function, iterable)
```

Example: Even Numbers

```
numbers = [1, 2, 3, 4, 5, 6]

result = filter(lambda x: x % 2 == 0, numbers)

print(list(result))
```

Output :

```
[2, 4, 6]
```

3. reduce() Function

reduce() reduces list into single value

Syntax :

```
from functools import reduce

reduce(function, iterable)
```

Example: Sum of Numbers

```
from functools import reduce

numbers = [1, 2, 3, 4]

result = reduce(lambda x, y: x + y, numbers)

print(result)
```

Output :

```
10
```

Q 10.Understanding how to create and access elements in a list.

A.A list is a collection of items stored in square brackets [ ].

Lists can store multiple values of different data types.

Creating a List

Example :

```
fruits = ["apple", "banana", "mango"]

print(fruits)
```

Output :

```
['apple', 'banana', 'mango']
```

List with Different Data Types

```
data = [10, "Python", 3.5, True]

print(data)
```

Output :

```
[10, 'Python', 3.5, True]
```

Accessing List Elements (Indexing)

Lists use **index numbers** starting from **0**

```
fruits = ["apple", "banana", "mango"]

print(fruits[0])

print(fruits[1])

print(fruits[2])
```

Output :

```
apple

banana

mango
```

Negative Indexing

```
fruits = ["apple", "banana", "mango"]
```

```
print(fruits[-1])

print(fruits[-2])
```

Output :

```
mango

banana
```

Changing List Elements

```
fruits = ["apple", "banana", "mango"]

fruits[1] = "orange"

print(fruits)
```

Output :

```
['apple', 'orange', 'mango']
```

List Length

```
fruits = ["apple", "banana", "mango"]

print(len(fruits))
```

Output :

```
3
```

Loop Through List

```
fruits = ["apple", "banana", "mango"]

for fruit in fruits:

  print(fruit)
```

Output :

```
apple

banana
```

mango

Q 11.Indexing in lists (positive and negative indexing).

A.Indexing means accessing elements of a list using position numbers.

Python supports:

- Positive Indexing
- Negative Indexing

Example List

fruits = ["apple", "banana", "mango", "orange"]

Positive Indexing :

Positive indexing starts from 0 (left to right)

Example :

print(fruits[0])

print(fruits[1])

print(fruits[2])

Output :

apple

banana

mango

Negative Indexing :

Negative indexing starts from -1 (right to left)

Example :

print(fruits[-1])

print(fruits[-2])

```
print(fruits[-3])
```

Output :

orange

mango

banana

Modify Using Index

```
fruits[1] = "grapes"

print(fruits)
```

Output :

['apple', 'grapes', 'mango', 'orange']

Q 12.Slicing a list: accessing a range of elements.

A.Slicing means accessing a range of elements from a list using start and end index.

Syntax :

```
list[start : end]
```

Example List :

```
numbers = [10, 20, 30, 40, 50, 60]
```

Basic Slicing :

```
print(numbers[1:4])
```

Output :

[20, 30, 40]

Slice from Beginning :

```
print(numbers[:3])
```

Output :

```
[10, 20, 30]
```

Slice Till End :

```
print(numbers[2:])
```

Output :

```
[30, 40, 50, 60]
```

Negative Slicing :

```
print(numbers[-4:-1])
```

Output :

```
[30, 40, 50]
```

Step Slicing :

```
print(numbers[0:6:2])
```

Output :

```
[10, 30, 50]
```

Reverse List Using Slicing :

```
print(numbers[::-1])
```

Output :

```
[60, 50, 40, 30, 20, 10]
```

Q 13. Common list operations: concatenation, repetition, membership.

A.1. Concatenation (+)

Concatenation is used to combine two lists.

Example :

```
list1 = [1, 2, 3]

list2 = [4, 5, 6]

result = list1 + list2

print(result)
```

Output :

```
[1, 2, 3, 4, 5, 6]
```

2. Repetition (*)

Repetition is used to **repeat list elements**

Example :

```
list1 = [1, 2, 3]

result = list1 * 2

print(result)
```

Output :

```
[1, 2, 3, 1, 2, 3]
```

3. Membership (in / not in)

Membership is used to **check if element exists in list**

Example :

```
list1 = [10, 20, 30]

print(20 in list1)

print(50 in list1)
```

Output :

True

False

Using not in :

```
list1 = [10, 20, 30]

print(40 not in list1)
```

Output :

True

Q 14.Understanding list methods like append(), insert(), remove(), pop().

A.1. append() Method :

append() is used to add an element at the end of list.

Example :

```
list1 = [10, 20, 30]

list1.append(40)

print(list1)
```

Output :

[10, 20, 30, 40]

2. insert() Method :

insert() is used to **add element at specific position**

Syntax :

list.insert(index, element)

Example :

```
list1 = [10, 20, 30]

list1.insert(1, 15)

print(list1)
```

Output :

[10, 15, 20, 30]

3. remove() Method :

remove() is used to **remove specific element**

Example :

```
list1 = [10, 20, 30, 40]

list1.remove(30)

print(list1)
```

Output :

[10, 20, 40]

4. pop() Method :

pop() is used to **remove element by index**

Example 1 (Without Index) :

```
list1 = [10, 20, 30]
```

```
list1.pop()

print(list1)
```

Output :

```
[10, 20]
```

Example 2 (With Index) :

```
list1 = [10, 20, 30]

list1.pop(1)

print(list1)
```

Output :

```
[10, 30]
```

Q 15.Iterating over a list using loops.

A.1. Using for Loop

This is the most common method.

Example :

```
list1 = [10, 20, 30, 40]

for i in list1:

   print(i)
```

Output :

```
10

20

30

40
```

2. Using while Loop

Example :

```
list1 = [10, 20, 30, 40]

i = 0

while i < len(list1):

   print(list1[i])

   i = i + 1
```

Output :

```
10

20

30

40
```

Q 16.Sorting and reversing a list using sort(), sorted(), and reverse().

A.1. sort() Method :

sort() is used to sort the original list permanently

Example :

```
list1 = [40, 10, 30, 20]

list1.sort()

print(list1)
```

Output :

```
[10, 20, 30, 40]
```

Sort in Descending Order :

```
list1 = [40, 10, 30, 20]

list1.sort(reverse=True)

print(list1)
```

Output :

```
[40, 30, 20, 10]
```

2. sorted() Function :

sorted() is used to **sort without changing original list**

Example :

```
list1 = [40, 10, 30, 20]

new_list = sorted(list1)

print("Original:", list1)

print("Sorted:", new_list)
```

Output :

```
Original: [40, 10, 30, 20]

Sorted: [10, 20, 30, 40]
```

Descending using sorted() :

```
list1 = [40, 10, 30, 20]

new_list = sorted(list1, reverse=True)

print(new_list)
```

Output :

```
[40, 30, 20, 10]
```

3. reverse() Method :

reverse() is used to **reverse list order**

Example :

```
list1 = [10, 20, 30, 40]

list1.reverse()

print(list1)
```

Output :

```
[40, 30, 20, 10]
```

Q 17.Basic list manipulations: addition, deletion, updating, and slicing.

A.1. Addition in List

You can add elements using append(), insert(), or +

Example :

```
list1 = [10, 20, 30]

list1.append(40)

print(list1)
```

Output :

```
[10, 20, 30, 40]
```

Using insert() :

```
list1 = [10, 20, 30]

list1.insert(1, 15)

print(list1)
```

Output :

[10, 15, 20, 30]

2. Deletion in List :

You can delete elements using:

- remove()
- pop()
- del

Using remove() :

```
list1 = [10, 20, 30]

list1.remove(20)

print(list1)
```

Output :

[10, 30]

Using pop() :

```
list1 = [10, 20, 30]

list1.pop()

print(list1)
```

Output :

[10, 20]

Using del :

```
list1 = [10, 20, 30]

del list1[1]

print(list1)
```

Output :

[10, 30]

3. Updating List Elements :

You can **update values using index**

Example :

```
list1 = [10, 20, 30]

list1[1] = 25

print(list1)
```

Output :

[10, 25, 30]

4. Slicing List :

Slicing is used to **access range of elements**

Example :

```
list1 = [10, 20, 30, 40, 50]

print(list1[1:4])
```

Output :

[20, 30, 40]

Reverse using slicing :

```
list1 = [10, 20, 30, 40]

print(list1[::-1])
```

Output :

[40, 30, 20, 10]

Q 18.Introduction to tuples, immutability.

A.A Tuple is a collection of elements that is ordered and immutable.

Creating a Tuple :

Tuples are created using **parentheses ( )**

Example :

```
tuple1 = (10, 20, 30)

print(tuple1)
```

Output :

```
(10, 20, 30)
```

Tuple with Different Data Types :

```
tuple1 = (10, "Python", 3.5, True)

print(tuple1)
```

Output :

```
(10, 'Python', 3.5, True)
```

Accessing Tuple Elements :

Using **index**

```
tuple1 = (10, 20, 30)

print(tuple1[0])

print(tuple1[2])
```

Output :

```
10

30
```

Q 19.Creating and accessing elements in a tuple.

A.A Tuple is a collection of items stored in parentheses ( ) and it is immutable.

Creating a Tuple :

Example :

```
tuple1 = (10, 20, 30, 40)

print(tuple1)
```

Output :

```
(10, 20, 30, 40)
```

Tuple with Different Data Types :

```
tuple1 = (10, "Python", 3.5, True)

print(tuple1)
```

Output :

```
(10, 'Python', 3.5, True)
```

Accessing Tuple Elements :

Tuple elements are accessed using **index**

Example :

```
tuple1 = (10, 20, 30, 40)

print(tuple1[0])

print(tuple1[2])
```

Output :

```
10

30
```

Negative Indexing :

```
tuple1 = (10, 20, 30, 40)

print(tuple1[-1])

print(tuple1[-2])
```

Output :

40

30

Accessing Using Loop :

```
tuple1 = (10, 20, 30)

for i in tuple1:

   print(i)
```

Output

10

20

30

Tuple Slicing :

```
tuple1 = (10, 20, 30, 40, 50)

print(tuple1[1:4])
```

Output :

(20, 30, 40)

Q 20.Basic operations with tuples: concatenation, repetition, membership.

A. 1. Concatenation (+) :

Concatenation is used to join two tuples

Example :

```
tuple1 = (10, 20, 30)

tuple2 = (40, 50, 60)

result = tuple1 + tuple2

print(result)
```

Output :

(10, 20, 30, 40, 50, 60)

2. Repetition (*) :

Repetition is used to **repeat tuple elements**

Example :

```
tuple1 = (1, 2, 3)

result = tuple1 * 2

print(result)
```

Output :

(1, 2, 3, 1, 2, 3)

3. Membership (in / not in) :

Membership is used to **check element exists or not**

Example :

```
tuple1 = (10, 20, 30)

print(20 in tuple1)

print(50 in tuple1)
```

Output :

True

False

Using not in :

tuple1 = (10, 20, 30)

print(40 not in tuple1)

Output :

True

Q 21.Accessing tuple elements using positive and negative indexing.

A.tuple1 = (10, 20, 30, 40, 50)

1. Positive Indexing :

Positive indexing **starts from 0** (left to right)

Index	Value
0	10
1	20
2	30
3	40

4	50

Example :

```
tuple1 = (10, 20, 30, 40, 50)

print(tuple1[0])

print(tuple1[2])

print(tuple1[4])
```

Output :

```
10

30

50
```

2. Negative Indexing :

Negative indexing **starts from -1** (right to left)

Index	Value
-1	50
-2	40
-3	30
-4	20

-5 10

Example :

```
tuple1 = (10, 20, 30, 40, 50)
print(tuple1[-1])
print(tuple1[-3])
print(tuple1[-5])
```

Output :

50

30

10

Access Using Loop :

```
tuple1 = (10, 20, 30)
for i in range(len(tuple1)):
  print(tuple1[i])
```

Output :

10

20

30

Q 22.Slicing a tuple to access ranges of elements.

A.Syntax :

tuple[start : end]

Example Tuple :

tuple1 = (10, 20, 30, 40, 50)

Basic Slicing :

print(tuple1[1:4])

Output :

(20, 30, 40)

Slice from Beginning :

print(tuple1[:3])

Output :

(10, 20, 30)

Slice Till End :

print(tuple1[2:])

Output :

(30, 40, 50)

Negative Slicing :

print(tuple1[-4:-1])

Output :

(20, 30, 40)

Step Slicing :

print(tuple1[0:5:2])

Output :

(10, 30, 50)

Reverse Tuple Using Slicing :

```
print(tuple1[::-1])
```

Output :

(50, 40, 30, 20, 10)

Q 23.Introduction to dictionaries: key-value pairs.

A.A Dictionary is a collection of key-value pairs used to store data.

Creating a Dictionary :

Dictionaries are created using **curly braces { }**

Example :

```
student = {
   "Name": "Sonali",
   "Age": 24,
   "Course": "Python"
}
print(student)
```

Output

{'Name': 'Sonali', 'Age': 24, 'Course': 'Python'}

Q 24.Accessing, adding, updating, and deleting dictionary elements.

A.student = {

"Name": "Sonali",

"Age": 24,

"Course": "Python"

}

1. Accessing Dictionary Elements :

Access values using **keys**

```
print(student["Name"])

print(student["Age"])
```

Output :

Sonali

24

2. Adding Dictionary Elements :

Add new key-value pair

```
student["City"] = "Surat"

print(student)
```

Output :

{'Name': 'Sonali', 'Age': 24, 'Course': 'Python', 'City': 'Surat'}

3. Updating Dictionary Elements :

Update value using key

```
student["Age"] = 23

print(student)
```

Output :

{'Name': 'Sonali', 'Age': 23, 'Course': 'Python'}

4. Deleting Dictionary Elements :

Using del :

```
del student["Course"]
```

```
print(student)
```

Output :

```
{'Name': 'Sonali', 'Age': 22}
```

Using pop() :

```
student.pop("Age")

print(student)
```

Output :

```
{'Name': 'Sonali'}
```

Delete Entire Dictionary :

```
del student
```

Q 25.Dictionary methods like keys(), values(), and items().

A.

```
student = {

"Name": "Sonali",

"Age": 24,

"Course": "Python",

"City": "Surat"

}
```

1. keys() Method :

The **keys()** method returns **all keys** in the dictionary.

Example :

```
print(student.keys())
```

Output :

```
dict_keys(['Name', 'Age', 'Course', 'City'])
```

2. values() Method :

The **values()** method returns **all values** in the dictionary.

Example :

```
print(student.values())
```

Output :

```
dict_values(['Sonali', 24, 'Python', 'Surat'])
```

3. items() Method :

The **items()** method returns **key-value pairs** as tuples.

Example :

```
print(student.items())
```

Output :

```
dict_items([('Name', 'Sonali'), ('Age', 24), ('Course', 'Python’), ('City', 'Surat')])
```

Q 26. Iterating over a dictionary using loops.

A.1. Iterating Through Keys (Default) :

```
student = {
"name": "Sonali",
"age": 22,
 "course": "Python"
}
for key in student:
   print(student.keys())
```

Output :

name

age

course

2. Iterating Through Values :

```
for value in student:
  print(student.values())
```

Output :

Sonali

22

Python

3. Iterating Through Key-Value Pairs :

```
for key, value in student.items():
  print(student.items())
```

Output

name : Sonali

age : 22

course : Python

Q 27.Merging two lists into a dictionary using loops or zip().

A.keys = ["name", "age", "course"]

values = ["Sonali", 22, "Python"]

```
result = {}

for i in range(len(keys)):

   result[keys[i]] = values[i]

print(result)
```

Q 28.Counting occurrences of characters in a string using dictionaries.

A.

```
string = "python"

count = {}

for char in string:

   if char in count:

       count[char] += 1

   else:

       count[char] = 1

print(count)
```

Output :

```
{'p': 1, 'y': 1, 't': 1, 'h': 1, 'o': 1, 'n': 1}
```

Q 29.Defining functions in Python.

A.A function is a block of code that performs a specific task.

Syntax :

```
def function_name():

   statements
```

Example :

```
def text():
   print("Hello Sonali")
text()
```

Output :

Hello Sonali

Q 30. Different types of functions: with/without parameters, with/without return values.

A.With Parameters :

```
def text(name):
   print("Hello", name)
text("Sonali")
```

Output :

Hello Sonali

Without Parameters :

```
def add():
   a = 10
   b = 20
   return a + b


result = add()
print(result)
```

Output

30

With Return Value :

```
def add(a, b):
  return a + b
result = add(10, 20)
print(result)
```

Output :

30

Without Return Value :

```
def text(name):
  print("Hello", name)
text("Sonali")
```

Output :

Hello Sonali

Q 31.Anonymous functions (lambda functions).

A. →A Lambda function is a small anonymous function (function without a name).

→It is created using the lambda keyword.

Example :

```
square = lambda x: x * x
print(square(5))
```

Output :

25

Lambda with map() :

```
n = [1, 2, 3, 4, 5]

r = list(map(lambda x: x * 2, n))

print(list(r))
```

Output :

```
[2, 4, 6, 8, 10]
```

Lambda with filter() :

```
n = [1, 2, 3, 4, 5, 6]

r = list(filter(lambda x: x % 2 == 0, n))

print(list(r))
```

Output :

```
[2, 4, 6]
```

Lambda with reduce() :

```
l = [10,20,30,40,50,60]

from functools import reduce

k = reduce(lambda x,y : max(x,y),l)

print(k)
```

Q 32.Introduction to Python modules and importing modules.

A.A module in Python is a file containing Python code (functions, variables, classes) that you can reuse in another program.

Example of a Module :

Create a file named **mymodule.py**

```
def greet(name):
  print("Hello", name)
def add(a, b):
  return a + b
```

Importing a Module :

Now create another file:

```
import mymodule
mymodule.greet("Sonali")
print(mymodule.add(10, 20))
```

Output :

```
Hello Sonali
30
```

Q 33.Standard library modules: math, random.

A.1. Math Module :

The math module provides mathematical functions like square root, power, factorial, etc.

Import Math Module :

```
import math
```

Common Math Functions :

1. Square Root :

```
import math
print(math.sqrt(25))
```

Output

5.0

2. Power :

```
import math

print(math.pow(2, 3))
```

Output :

8.0

3. Factorial :

```
import math

print(math.factorial(5))
```

Output :

120

4. Ceiling and Floor :

```
import math

print(math.ceil(4.2))

print(math.floor(4.8))
```

Output :

5

4

5. Value of Pi :

```
import math
```

```
print(math.pi)
```

Output :

3.141592653589793

2. Random Module :

The **random module** is used to generate random numbers.

Import Random Module :

```
import random
```

Random Functions :

1. Random Integer :

```
import random

print(random.randint(1, 10))
```

Output (random) :

7

2. Random Float :

```
import random

print(random.random())
```

Output (random) :

0.5678

3. Random Choice :

```
import random

list1 = [10, 20, 30, 40]

print(random.choice(list1))
```

Output (random) :

30

4. Shuffle List :

```
import random

list1 = [1, 2, 3, 4, 5]

random.shuffle(list1)

print(list1)
```

Output (random) :

[3, 1, 5, 2, 4]

Q 34.Creating custom modules.

A.Step 1: Create a Module File :

Create a file named mymodule.py

```
def text(name):

   print("Hello", name)

def add(a, b):

   return a + b

pi = 3.14
```

Step 2: Import the Module :

Create another file main.py

```
import mymodule

mymodule.text("Sonali")

print(mymodule.add(10, 20))

print(mymodule.pi)
```

Output :

Hello Sonali

30

3.14